L04 - Dronagiri: The Last Sanjeevani

PINAKA · Forensics · 2026-11-01 · flag format PINAKA{...}

Flag: `PINAKA{dronagiri_the_last_sanjeevani}`

Summary

The challenge hands over a dead replicated key-value store: three snapshot segments, three damaged write-ahead logs, a spectrometer calibration file and a terrain DEM, all in undocumented binary formats. Getting the flag meant reversing every format, rebuilding the committed history with a 2-of-3 majority rule (ignoring torn, zeroed and single-node writes), decoding the spectra (an integer Haar wavelet), calibrating the wavelength axis from two lamp lines, and then finding the one committed specimen that both glows in 1108-1123 nm and sits on a strict local maximum of the terrain between 3000 and 3400. That record went to the validator, which returned the flag.

Step 1 - Recon

Every file starts with an 8-byte ASCII magic (`PNKDSEG1`, `PNKDWAL1`, `PNKCAL01`, `PNKDEM01`). After the magic comes a small header, closed off by a CRC-32 of the header bytes. That CRC is what let me confirm each layout guess. Two things stood out early:

- The `u32` straight after the version differs per file: `0x0001 / 0x10001 / 0x20001`. That's a `u16` version plus a `u16` **node id**.
- Each WAL header declares a frame count that is 7 higher than the number of `DREC` magics in the file. Seven frames per node had been wiped. They turned out to be a run of zero bytes near the end of each log (the "overwritten tail span").

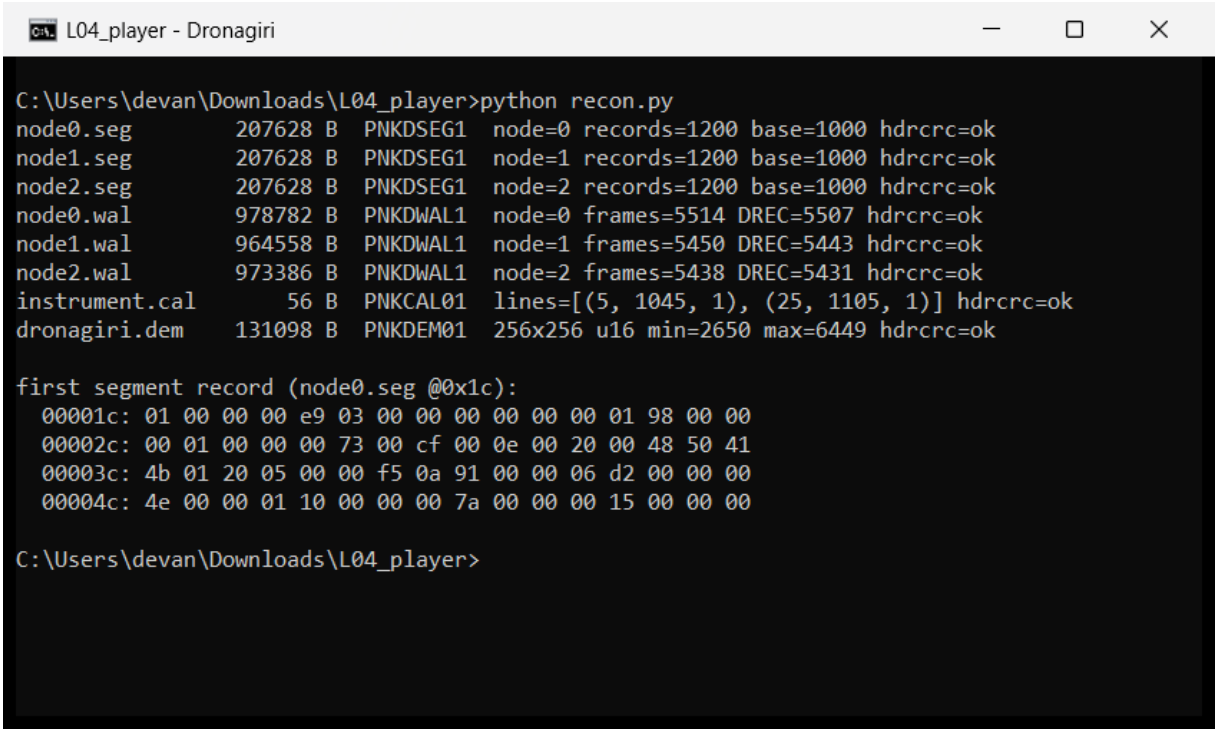


Figure 1 – 01_recon.png

The layouts I settled on (all little-endian unless noted):

Structure	Layout
SEG header	`magic[8] ver:u16 node:u16 count:u32 base:u64 crc32:u32`

Structure	Layout
SEG record	`key:u32 seqno:u64 op:u8 vlen:u32 value[vlen] crc32(record):u32`
WAL header	`magic[8] ver:u16 node:u16 nframes:u32 crc32:u32`
WAL frame	`"DREC" blen:u32 body[blen] crc32("DREC"+blen+body):u32`
WAL body	`lsn:u64 seqno:u64 op:u8 key:u32 vlen:u32 value[vlen]` (op 1 = put, 2 = delete)
value (152 B)	`key:u32 x:u16 y:u16 z:u16 0x20 0x00 "HPAK" 01 20 05 00 crc32(coefs):u32 coef[32]`
CAL	header + `n` lines of `index:u16 num:i64 den:i64`
DEM	`magic ver w:u16 h:u16 min:u32 max:u32 crc32` + `w*h u16` elevations, row-major

The WAL body cost me the most time. I first read the first `u64` as the sequence number, but the same value pointed at completely different writes on each node. It is actually the **local log position (LSN)**: frames are stored out of order in the file, and each node numbers its own log `1..nframes`. The second `u64` is the real, cluster-wide `seqno`.

Step 2 - Reading the damage and the quorum

With the frame parser in place I put every CRC-valid write into a table keyed on `(key, seqno, op, value bytes)` and recorded which nodes hold an identical copy:

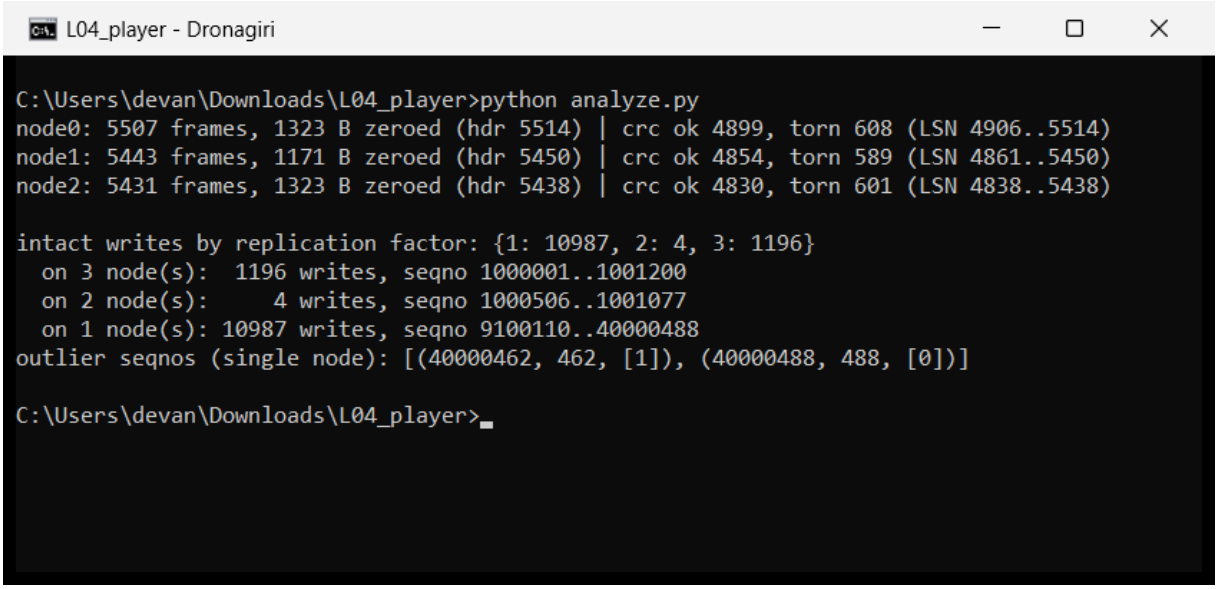


Figure 2 – 02_wal_quorum.png

That one table explains the whole history:

- **Torn frames** (bad CRC) are exactly the last ~600 LSNs on every node. Diffing them shows single-byte flips (`HPAK` -> `H\nAK`, `0x20` -> `0x5a`, garbage in the LSN). They never match anything on another node, so they can't be repaired by voting and can't be committed. I dropped them.
- **1196 writes are on all three nodes and 4 more are on two** (the third copy is damaged). These 1200 writes are the post-checkpoint commits. Each specimen `k` gets seqno `1,000,000 + k`, which supersedes its snapshot version (seqno `1000 + k`). 20 of them are deletes.
- **10987 writes exist on one node only.** None is committed, however new its seqno looks. Two of them are obvious traps at seqno `40000462` / `40000488`. One of those, specimen 462 on node 1, glows and sits on a summit, so a solver that just takes the "latest version" lands on it.

So the committed archive is: for each key, the highest-seqno version present on >= 2 nodes, with committed deletes removed. That leaves **1180 live records**.

Step 3 - Decoding, calibrating, cross-referencing

Spectra. The 32 words after the `HPAK` header are big-endian sign-magnitude `int32`s (bit 31 is the sign). The CRC tag in front of them is just `crc32` of those 128 bytes. The header `01 20 05` reads as version 1, 32 bands, 5 levels. 32 = 2^5, and the coefficients roughly halve as you move through the array, which pointed to a 5-level Haar transform. The layout is `[approx, 16 finest details, 8, 4, 2, 1]`, and the integer S-transform inverts it:

```
def inv_haar(c, levels=5):
    sizes = [len(c) >> (k + 1) for k in range(levels)]        # 16, 8, 4, 2, 1
    blocks, i = [], 1
    for sz in sizes:
        blocks.append(c[i:i + sz]); i += sz
    s = [c[0]]
    for d in reversed(blocks):
        nxt = []
        for sv, dv in zip(s, d):
            a = sv - (dv >> 1)          # even sample (floor division)
            nxt += [a, a + dv]          # odd = even + detail
        s = nxt
    return s
```

I tried the opposite detail sign first (`d = even - odd`), and it gives negative ~-45000 spikes on a 3300 baseline. That can't be right. The version above gives a flat 80-420 background with one sharp peak per specimen.

Calibration. `instrument.cal` has two lamp lines: band 5 -> 1045/1 nm and band 25 -> 1105/1 nm. The axis is linear, so `p = 60/20 = 3`, `q = 1045 - 15 = 1030`, i.e. **nm = 3*i + 1030**. The 32 bands cover 1030-1123 nm, and the luminescence window 1108-1123 nm is bands 26-31.

Terrain. The DEM has 11 cells that are strictly higher than all 8 neighbours with elevation in 3000-3400. Indexing as `y*256 + x` gives a hit and the transposed order gives none, which confirms row-major storage.

Cross-reference. Of the 1180 committed records, exactly one has its peak in the window *and* sits on one of those 11 summits. `solve.py` asserts that the match is unique:

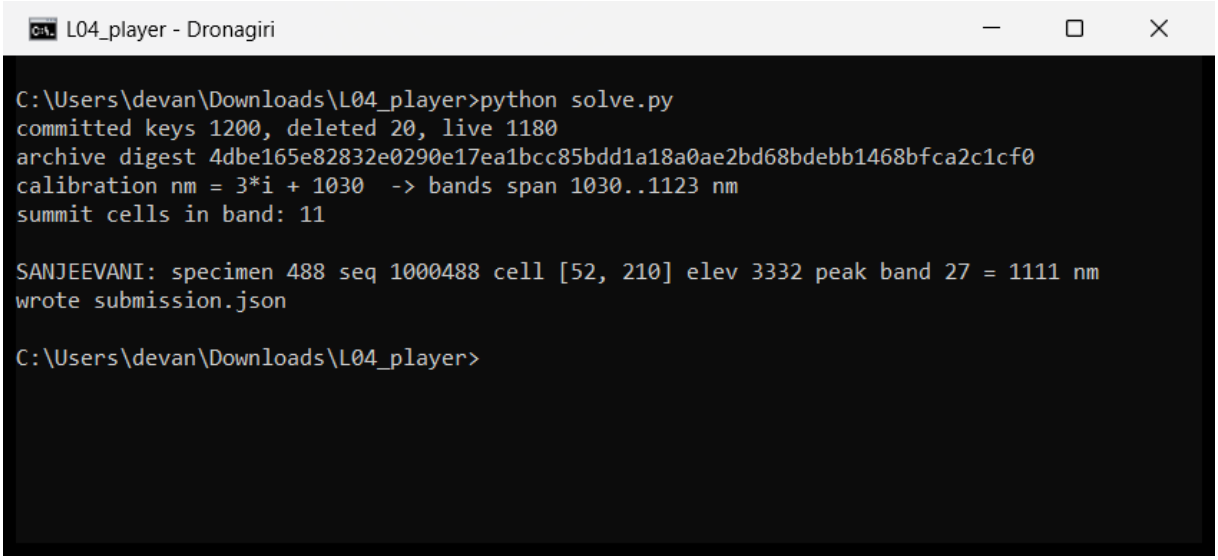


Figure 3 – 03_solve.png

The full solve script (`dronagiri.py` holds the parsers and the Haar decoder):

```
import hashlib, json, sys, urllib.request
from collections import defaultdict
from fractions import Fraction
from dronagiri import decode, read_cal, read_dem, read_seg, read_wal

VALIDATOR = "https://pinaka-l04-cyber-xo-x.vercel.app/dronagiri/sanjeevani"
WINDOW = (1108, 1123)
BAND = (3000, 3400)

copies = defaultdict(set)
for n in range(3):
    for key, seq, op, val in read_seg(n)["recs"]:
        copies[(key, seq, op, val)].add(n)
    for f in read_wal(n)["frames"]:
        if f["ok"]:
            copies[(f["key"], f["seq"], f["op"], f["val"])].add(n)

latest = {}
for (key, seq, op, val), nodes in copies.items():
    if len(nodes) >= 2 and (key not in latest or seq > latest[key][0]):
        latest[key] = (seq, op, val)
archive = {k: v for k, v in latest.items() if v[1] == 1}
digest = hashlib.sha256(";".join(
    f"{k}:{archive[k][0]}:{hashlib.sha256(archive[k][2]).hexdigest()}"
    for k in sorted(archive)).encode()).hexdigest()

(i0, n0, d0), (i1, n1, d1) = read_cal()
p = (Fraction(n1, d1) - Fraction(n0, d0)) / (i1 - i0)
q = Fraction(n0, d0) - p * i0

W, H, _, _, E = read_dem()
el = lambda x, y: E[y * W + x]
summits = {(x, y) for y in range(1, H - 1) for x in range(1, W - 1)
           if BAND[0] <= el(x, y) <= BAND[1]
           and all(el(x + dx, y + dy) < el(x, y)
                   for dx in (-1, 0, 1) for dy in (-1, 0, 1) if dx or dy)}

hits = []
for key, (seq, op, val) in archive.items():
    d = decode(val)
    pb = max(range(32), key=d["spec"].__getitem__)
    nm = p * pb + q
    if WINDOW[0] <= nm <= WINDOW[1] and (d["x"], d["y"]) in summits:
        hits.append((key, seq, d, pb, nm))
assert len(hits) == 1, hits
key, seq, d, pb, nm = hits[0]

sub = {"specimen_id": key, "commit_seqno": seq, "cell": [d["x"], d["y"]],
       "elevation": el(d["x"], d["y"]), "peak_band": pb,
       "peak_wavelength": [nm.numerator, nm.denominator],
       "wavelength_solution": [p.numerator, p.denominator, q.numerator, q.denominator],
       "restored_spectrum": d["spec"], "committed_record_count": len(archive),
       "committed_archive_digest": digest}
json.dump(sub, open("submission.json", "w"))
if "--submit" in sys.argv:
    req = urllib.request.Request(VALIDATOR, data=json.dumps(sub).encode(),
                                 headers={"Content-Type": "application/json"})
    print(urllib.request.urlopen(req, timeout=30).read().decode())
```

(The file in the repo has the same logic plus progress prints.)

Step 4 - Verify

Before submitting I wanted independent checks on each part of the answer. `verify.py` checks:

- **All 16978** decodable values pass their coefficient CRC, and re-encoding the decoded spectrum with the forward S-transform reproduces the stored coefficients exactly, with no negative bands anywhere.
- Specimen 488's only multi-node version is `1000488`, which sits on all three nodes. The 11 newer versions, including the `40000488` decoy, are each on a single node.
- Cell (52, 210) is 3332 and strictly above its eight neighbours (max 3329).
- The peak is band 27 at 57817 counts: 3*27 + 1030 = **1111 nm**, inside the window.

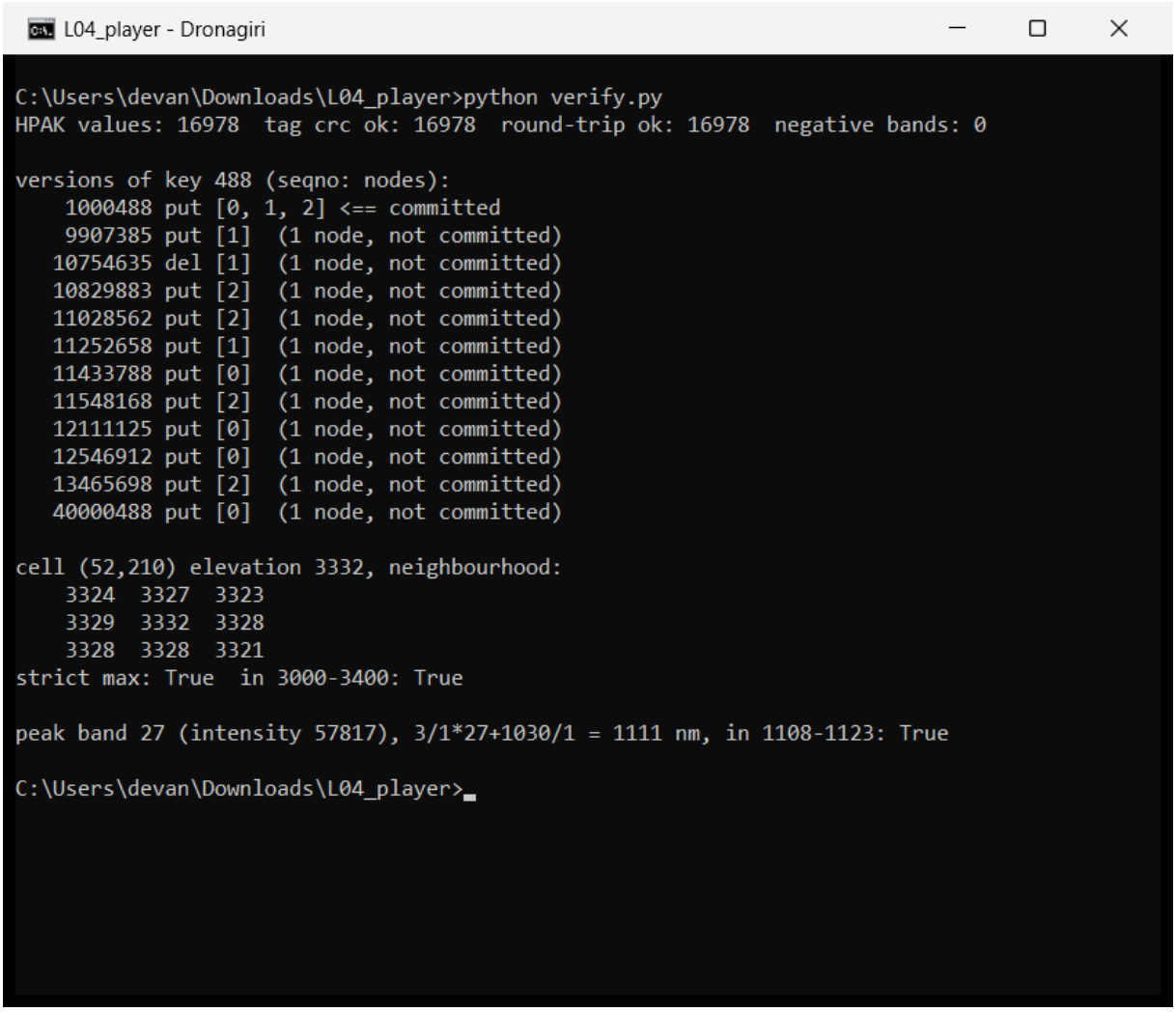


Figure 4 – 04_verify.png

Step 5 - Get the flag

`submission.json` posted to the validator endpoint:

```
curl -s -H "Content-Type: application/json" -d @submission.json \
  https://pinaka-l04-cyber-xo-x.vercel.app/dronagiri/sanjeevani | python -m json.tool
```

Submitted record:

field	value
specimen_id	488
commit_seqno	1000488
cell / elevation	[52, 210] / 3332
peak_band / peak_wavelength	27 / [1111, 1]
wavelength_solution	[3, 1, 1030, 1]
committed_record_count	1180
committed_archive_digest	`4dbe165e82832e0290e17ea1bcc85bdd1a18a0ae2bd68bdebb1468bfca2c1cf0`

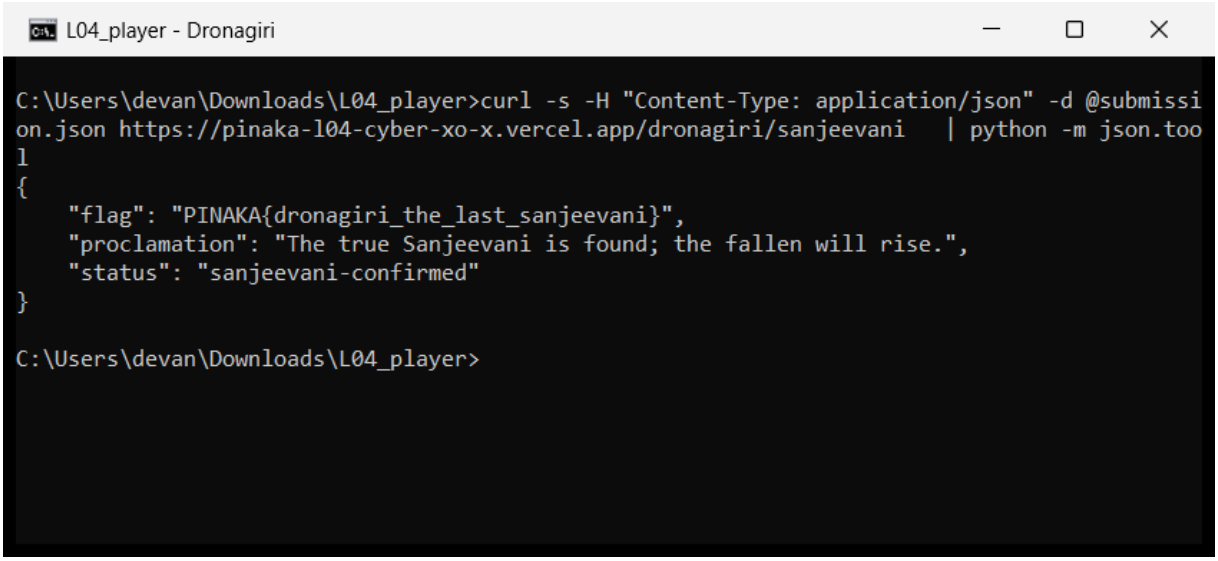


Figure 5 – 05_flag.png

Flag

```
PINAKA{dronagiri_the_last_sanjeevani}
```

Files

File	Purpose
`dronagiri.py`	Parsers for SEG / WAL / CAL / DEM, HPAK value decoder, forward + inverse integer Haar
`recon.py`	Step 1: magics, header fields, header CRC checks, first segment record hexdump
`analyze.py`	Step 2: WAL damage summary and replication-factor table
`solve.py`	Step 3: quorum reconciliation, calibration, summits, unique match, writes `submission.json` (`--submit` posts it)
`verify.py`	Step 4: decoder round-trip, version history of the answer key, DEM neighbourhood, wavelength check
`submission.json`	The exact record submitted to the validator
`build_pdf.py`	Builds `Dronagiri_writeup.pdf` from this file
`screenshots/01_recon.png`	`python recon.py`
`screenshots/02_wal_quorum.png`	`python analyze.py`
`screenshots/03_solve.png`	`python solve.py`
`screenshots/04_verify.png`	`python verify.py`
`screenshots/05_flag.png`	validator response via `curl`